

Acta de daily 03/09

Una por ceremonia, en `scrum/actas`. El criterio de la rúbrica es que los acuerdos sean **accionables**: con responsable y fecha.

Acta de daily

Campo	Contenido
Fecha y hora	03/09/26, -----14:45
Duración	20 mins
Asistentes	Bastian Almonacid, Andres Alvarez, Enrique Pincheira, Sebastian Sandoval, Vicente Salazar
Ausentes justificados	Belen Bravo
Facilitador (SM)	Vicente Salazar

Temas tratados

Check de progreso: D1.T1, D1.T2, X2.T1 completados Reunion de integracion: (Docker ports, Integración con el módulo 1) Mod.3 Aún no termina su autenticación. Mod 1.

Acuerdos

Cada acuerdo con responsable y fecha. Un acuerdo sin responsable no es un acuerdo.

#	Acuerdo	Responsable	Fecha comprometida
1	Continuar con tareas asignadas en sesión anterior, documentadas en trello.	Todo el equipo M4	3/09/2026
2	Entrega de la autenticación Mod. 3	Mod. 3	07/09/2026
3	Entrega de SP de Mod. 1	Mod. 1	07/09/2026

Impedimentos declarados

Impedimento	Depende de	Desde	Estado
Declaración de puertos	Mod 1, Mod 2, Mod 3, Mod 4	03/09	No listo
Implementaciones del Mod. 1	Mod. 1	03/09	No listo
Implementaciones de autenticación Mod. 3	Mod. 3	03/09	No listo

Declaración de Uso de IA Generativa

Archivo `USO-IA.md` en la raíz del repositorio, actualizado por sprint, y sección equivalente en el informe final. Si el equipo no usó IA, se declara igual.

Sprint	Herramienta	En qué se usó	Verificación humana aplicada	Responsable
S2	Claude	Borrador del consumidor idempotente de reserva.creada	Se ejecutaron las pruebas de duplicados y se revisó en code review	M. Rojas

S1	Codex	Para implementar Prompt usado: Quiero implementar autenticación JWT mediante middleware en mi API Gateway. Contexto: * El Gateway es el punto de entrada único hacia los distintos módulos/microservicios. * La autenticación debe realizarse en el Gateway antes de enrutar la petición. * Los clientes enviarán el JWT mediante el header: Authorization: Bearer <token> Objetivo: Implementar un middleware de autenticación que intercepte las rutas protegidas, valide el JWT y permita continuar la petición solamente cuando el token sea válido. Requisitos funcionales: 1. Crear un middleware de autenticación JWT. 2. El middleware debe: * Leer el header `Authorization`. * Verificar que exista. * Verificar que utilice el formato `Bearer <token>`. * Extraer únicamente el token. * Validar la firma del JWT. * Validar la fecha de expiración (`exp`). * Validar los claims adicionales necesarios según la configuración actual del proyecto, como `issuer` o `audience`, solamente si estos ya forman parte del sistema. * Extraer del token información útil del usuario, como `sub`, `user_id`, `role` u otros claims existentes. * Adjuntar esa información al contexto/request para que pueda ser utilizada posteriormente. 3. Si el token no existe, responder: * HTTP 401 Unauthorized. 4. Si el token tiene un formato incorrecto, responder: * HTTP 401 Unauthorized.	Se verifico todo pasando 14 test antes de mergear algo.	Bastian Almonacid
----	-------	--	--	-------------------

	5. Si el token es inválido, está alterado o su firma no es válida: * HTTP 401 Unauthorized. 6. Si el token está expirado: * HTTP 401 Unauthorized. 7. Si el token es válido: * Continuar con la petición. * Permitir que el Gateway enrute la petición hacia el servicio correspondiente. 8. Deben existir rutas públicas que no pasen por el middleware de autenticación, por ejemplo: * login * health check * documentación de API, si actualmente existe No inventes rutas nuevas. Identifica primero cuáles rutas existentes deberían ser públicas y cuáles protegidas según la estructura actual del proyecto. Arquitectura esperada: <pre> Cliente Authorization: Bearer <JWT> v API Gateway --> Middleware JWT --> inválido --> 401 --> válido v Routing v Microservicio </pre> Consideraciones de seguridad: * No guardar el JWT completo en logs. * No exponer secretos, claves privadas ni información sensible en errores. * Utilizar variables de entorno o el mecanismo de configuración ya utilizado	
--	---	--

		por el proyecto para claves, secretos, issuer, audience, etc. * No hardcodear secretos. * No aceptar algoritmos JWT arbitrariamente. Utilizar únicamente el algoritmo configurado por el sistema. * No confiar en datos del payload antes de validar correctamente la firma. * Mantener separación entre autenticación y autorización. * Por ahora el middleware debe autenticar al usuario. Si ya existe autorización por roles/permisos, reutilizarla; de lo contrario, no implementar un sistema complejo de permisos innecesariamente. Rate limiting: La historia de usuario también requiere rate limiting. Revisa si ya existe una implementación. * Si existe, intégrala correctamente con el flujo del middleware. * Si no existe, indica claramente dónde debería implementarse, pero mantén separada la responsabilidad de autenticación y rate limiting. Pruebas: Implementar tests para comprobar como mínimo: 1. Petición sin header Authorization -> 401. 2. Authorization sin `Bearer` -> 401. 3. JWT inválido -> 401. 4. JWT modificado/firma inválida -> 401. 5. JWT expirado -> 401. 6. JWT válido -> la petición continúa. 7. Una ruta pública puede accederse sin JWT. 8. Una ruta protegida no puede accederse sin JWT. 9. Con JWT válido, el Gateway continúa haciendo routing correctamente hacia el módulo correspondiente. Antes de modificar código: 1. Analiza la estructura actual del Gateway. 2. Identifica: * framework utilizado;	
--	--	---	--

		* archivo de entrada; * configuración de rutas; * mecanismo actual de routing/proxy; * librerías JWT existentes; * configuración mediante variables de entorno; * tests existentes; * sistema de manejo de errores. 3. Reutiliza la arquitectura y convenciones actuales. 4. No agregues dependencias nuevas si ya existe una librería apropiada en el proyecto. Implementación: * Mantén el código simple, legible y modular. * Evita sobreingeniería. * No dupliques lógica. * Respeta las convenciones del repositorio. * Usa nombres claros. * Si el proyecto utiliza docstrings, utiliza Google Style. * No agregues comentarios innecesarios que simplemente repitan lo que hace el código. Al finalizar: 1. Muéstrame qué archivos modificaste o creaste. 2. Explica brevemente la responsabilidad de cada archivo. 3. Explica el flujo completo desde que llega una petición hasta que se enruta al servicio. 4. Indica cómo configurar las variables de entorno necesarias. 5. Indica cómo ejecutar los tests. 6. Señala cualquier decisión de arquitectura relevante que hayas tenido que tomar. Criterios de aceptación finales: * El Gateway valida JWT. * Peticiones no autenticadas son rechazadas. * Las rutas públicas siguen funcionando sin autenticación. * Las peticiones autenticadas continúan hacia el módulo correspondiente.	
--	--	--	--

		* La autenticación está centralizada en un middleware. * Los secretos no están hardcoded. * Existen pruebas de autenticación y routing. * La implementación respeta la arquitectura existente del repositorio. Para realizar esto crea una branch nueva y realiza todo ahí para ver si funciona y luego revisar para el merge		
S1	CODEX	# Tarea: Configurar Gateway/BFF para PubTube — US-D1 Necesito implementar la tarea técnica: "Configurar gateway/BFF (Nginx/Traefik o servicio propio)" correspondiente a: US-D1 · API Gateway con autenticación JWT [8 SP · Must] IMPORTANTE: Esta tarea NO corresponde a implementar toda US-D1. Solo debes implementar la base arquitectónica/configuración del Gateway/BFF. Las siguientes subtareas se realizarán por separado y NO deben implementarse completamente ahora: - Validación JWT y middleware de autenticación. - Rate limiting. - Autorización/RBAC. - Observabilidad completa. - Métricas Prometheus. - OpenTelemetry. - Integración completa con los módulos. - Pruebas completas de autenticación. - Docker / docker-compose. ## Contexto arquitectónico Tras revisar el repositorio se decidió utilizar una arquitectura híbrida: Frontend ↓ NGINX ↓ FastAPI Gateway/BFF ↓ Módulo 1 / Módulo 2 / Módulo 3 Responsabilidades: NGINX: - Reverse proxy de entrada. - Recibir tráfico externo. - Reenviar las solicitudes al Gateway FastAPI. - Preparar soporte para WebSocket/SSE. - Propagar headers HTTP necesarios.	Se verificó todo realizando test.	Sebastián Sandoval

		FastAPI Gateway/BFF: - Mantener la lógica de aplicación del gateway. - Centralizar el routing hacia módulos internos. - Futuramente validar JWT. - Futuramente realizar agregación de respuestas. - Futuramente normalizar contratos. - Mantener OpenAPI. - Gestionar correlation IDs. La intención NO es reemplazar FastAPI con NGINX. NGINX será únicamente la capa externa de infraestructura/proxy. ## Estado conocido del repositorio Según la revisión previa: - Existe un gateway FastAPI mínimo en: gateway/app - Actualmente existe: GET /api/health - Las rutas futuras están señaladas en: gateway/app/api/router.py - Existe configuración de módulos/JWT en: gateway/.env.example - Actualmente no existe configuración NGINX. - Las variables: MODULE1_URL MODULE2_URL MODULE3_URL existen o están contempladas, pero todavía no tenemos contratos/nombres/puertos definitivos de todos los módulos. IMPORTANTE: No inventes contratos ni endpoints upstream que todavía no existan. ## Restricción importante de trabajo en equipo NO modifiques: - docker-compose.yml - Dockerfiles - configuración de redes Docker - servicios Docker - volúmenes Docker Otro integrante está encargado específicamente de Docker/X2. Si para que NGINX funcione posteriormente Docker necesita configuración adicional, DOCUMENTA	
--	--	---	--

		claramente qué necesita el compañero, pero no lo implementes. --- # Objetivo Dejar preparado un Gateway/BFF real y extensible basado en: NGINX → FastAPI → módulos internos sin implementar todavía toda la funcionalidad de US-D1. La solución debe ser simple, mantenible y adecuada al alcance académico del proyecto. --- # Paso 1 — Auditar antes de modificar Primero inspecciona el repositorio. Revisa específicamente: - gateway/ - gateway/app/ - gateway/app/main.py - gateway/app/api/ - gateway/app/api/router.py - gateway/app/core/ - gateway/.env.example - requirements / pyproject / dependencias existentes - README - ADR/ - tests/ Determina: 1. Cómo está estructurado actualmente FastAPI. 2. Cómo se cargan las variables de entorno. 3. Cómo está implementado `/api/health`. 4. Si ya existe un patrón para services/clients/config. 5. Si ya existe código relacionado con proxy HTTP. 6. Si ya existe manejo de correlation ID. 7. Si existe alguna convención establecida por archivos del repositorio, especialmente: agents/software-engineering-loop.md Respetar la arquitectura existente. No reorganices carpetas innecesariamente. --- # Paso 2 — Crear configuración de NGINX Crear una configuración NGINX preparada para actuar como entrada del sistema. Ubicación recomendada: nginx/templates/default.conf.template	
--	--	---	--

		Si la estructura del repositorio justifica otra ubicación, puedes adaptarla, pero explica el motivo. La configuración debe: - recibir tráfico HTTP; - reenviar las solicitudes <code>/api/...</code> al Gateway FastAPI; - preservar: - Host - X-Real-IP - X-Forwarded-For - X-Forwarded-Proto - X-Correlation-Id, cuando exista; - preparar soporte HTTP/1.1; - permitir Upgrade/Connection para futuro WebSocket; - configurar timeouts razonables para operaciones normales; - contemplar que posteriormente existirán cargas de video grandes. No implementar todavía una configuración extremadamente compleja. No agregar: - TLS/HTTPS real; - certificados; - OAuth; - WAF; - caching complejo; - balanceadores múltiples; - rate limiting completo si corresponde a otra subtask. Si alguna de estas cosas resulta recomendable, dejarla documentada como trabajo futuro. --- # Paso 3 — Preparar FastAPI como BFF Actualmente FastAPI no debe ser solamente un <code>/api/health</code>. Crear una estructura mínima que permita posteriormente enrutar peticiones hacia M1, M2 y M3. La solución debería separar conceptualmente: API ↓ BFF / service ↓ HTTP client ↓ upstream module Evita implementar lógica directamente en los routers si puede aislarse fácilmente. Ejemplo conceptual, NO obligatorio: <pre>gateway/app/ ├── api/ ├── core/ ├── clients/ └── services/</pre>	
--	--	--	--

		pero NO reorganices todo el proyecto únicamente para seguir este ejemplo. Adáptate a la arquitectura existente. --- # Paso 4 — Cliente HTTP para módulos Si actualmente no existe, preparar un cliente HTTP reutilizable para futuras llamadas hacia: MODULE1_URL MODULE2_URL MODULE3_URL Preferir: httpx.AsyncClient porque el Gateway está construido con FastAPI y las llamadas deberían ser asíncronas. La implementación debe: - leer URLs desde configuración; - evitar URLs hardcodeadas; - soportar headers reenviables; - permitir propagar `X-Correlation-Id`; - definir timeouts explícitos; - cerrar correctamente el cliente; - facilitar manejo posterior de errores upstream. NO inventar todavía endpoints internos. NO implementar un proxy genérico inseguro del tipo: /proxy/{url} El Gateway debe mantener rutas explícitas/controladas. --- # Paso 5 — Correlation ID El proyecto requiere propagación de correlationId extremo a extremo. Para esta tarea deja preparada la infraestructura mínima. Comportamiento recomendado: 1. Si llega `X-Correlation-Id`, conservarlo. 2. Si no llega, FastAPI puede generar un UUID v4 válido. 3. Adjuntarlo a la respuesta. 4. Dejar disponible ese valor para futuras llamadas upstream. IMPORTANTE: NGINX `request_id` NO debe considerarse automáticamente equivalente al correlationId del dominio, porque no garantiza necesariamente el UUID v4 requerido.	
--	--	---	--

		FastAPI debe seguir siendo responsable del identificador de correlación de la aplicación. Si implementar esto requiere introducir demasiado alcance, crear únicamente el middleware/base necesaria, sin instrumentar todavía métricas ni tracing. --- # Paso 6 — Healthcheck Mantener funcionando: GET /api/health No romper el contrato actual. Si resulta razonable, estructurar el healthcheck de forma que más adelante pueda incluir estado de upstreams, pero NO agregar dependencias reales hacia M1/M2/M3 todavía si estos módulos no están disponibles. El healthcheck básico debe continuar funcionando incluso cuando los otros equipos todavía no tengan sus servicios levantados. --- # Paso 7 — Variables de entorno Revisar `gateway/.env.example`. Agregar únicamente las variables necesarias para esta arquitectura si no existen. Ejemplo conceptual: MODULE1_URL= MODULE2_URL= MODULE3_URL= No agregar secretos reales. No colocar credenciales. No hardcodear IPs o localhost dentro de la lógica de aplicación. IMPORTANTE: Como Docker será realizado por otro integrante, no asumir nombres Docker definitivos como: module1 module2 module3 si esos nombres todavía no están acordados. Las URLs deben ser configurables. --- # Paso 8 — Manejo básico de errores upstream		
--	--	--	--	--

	Preparar una estrategia mínima para que cuando posteriormente se invoquen servicios internos puedan distinguirse: - timeout; - connection error; - upstream unavailable; - HTTP error upstream. No hace falta implementar todavía toda la especificación de errores del sistema. Evitar filtrar stack traces internos al frontend. No romper el formato de respuesta acordado por PubTube: Éxito: <pre>{ "status": "ok", "data": {} }</pre> Error: <pre>{ "status": "error", "code": "...", "message": "..." }</pre> No inventar nuevos contratos públicos innecesariamente. --- # Paso 9 — Tests Agregar pruebas exclusivamente para la infraestructura creada en esta tarea. Como mínimo evaluar: 1. `/api/health` continúa funcionando. 2. correlation ID recibido se conserva. 3. correlation ID ausente genera UUID v4 válido. 4. correlation ID aparece en la respuesta. 5. configuración del Gateway puede cargarse sin depender de módulos externos reales. Si se implementa un HTTP client abstraído, usar mocks para verificar: - propagación de `X-Correlation-Id`; - timeout configurado; - construcción correcta de la URL base. NO levantar M1/M2/M3 reales. NO modificar Docker para las pruebas. --- # Paso 10 — Documentación técnica Si el repositorio ya tiene ADR/, crear un ADR para documentar la decisión: NGINX como reverse proxy externo + FastAPI como BFF interno.		
--	--	--	--

		El ADR debe explicar como mínimo: Contexto: PubTube necesita un API Gateway y FastAPI ya existe como base del módulo. Alternativas consideradas: 1. NGINX como único gateway. 2. FastAPI como único gateway. 3. NGINX + FastAPI BFF. 4. Traefik + FastAPI, si corresponde. Decisión: NGINX + FastAPI. Justificación: NGINX: - proxy/infrastructure concerns. FastAPI: - lógica BFF; - JWT; - agregación; - contratos; - correlation ID; - OpenAPI. Consecuencias: - una capa adicional; - separación clara de responsabilidades; - configuración adicional; - facilita crecimiento futuro. NO inventar decisiones que el repositorio contradiga. --- # Fuera de alcance NO implementar en esta tarea: - autenticación JWT completa; - roles; - RBAC; - rate limiting completo; - Redis; - Prometheus; - Grafana; - OpenTelemetry; - RabbitMQ; - lógica de notificaciones; - dashboard; - endpoints de M1/M2/M3 que no estén acordados; - integración real con módulos inexistentes; - Docker Compose; - Dockerfiles; - TLS; - Kubernetes; - API Gateway cloud. No cambies código ajeno que no sea necesario para esta tarea. --- # Requisitos de calidad		
--	--	--	--	--

		La implementación debe: - seguir las convenciones actuales del repositorio; - usar async/await correctamente; - evitar duplicación; - mantener responsabilidades separadas; - usar configuración por variables de entorno; - no contener secretos; - ser fácil de extender; - mantener FastAPI como framework del BFF; - respetar los contratos existentes; - no inventar contratos de otros equipos. --- # Validación final Antes de terminar: 1. Ejecuta los tests existentes. 2. Ejecuta los tests nuevos. 3. Ejecuta formatter/linter configurado en el proyecto. 4. Verifica que `/api/health` siga funcionando. 5. Verifica que no hayas modificado docker-compose. 6. Verifica que no hayas agregado secretos. 7. Revisa el diff completo. 8. Detecta cambios accidentales o fuera de alcance. --- # Entrega Al terminar NO hagas commit automáticamente. Entrega un informe con: ## Archivos creados Lista + propósito. ## Archivos modificados Lista + cambio realizado. ## Arquitectura resultante Explica brevemente: Frontend → NGINX → FastAPI BFF → módulos ## Funcionalidad implementada Qué quedó funcionando realmente. ## Funcionalidad preparada pero no implementada Por ejemplo JWT, rate limiting, routing real, métricas. ## Dependencias con Docker	
--	--	---	--

		Lista exactamente qué deberá configurar el compañero encargado de Docker para integrar NGINX con FastAPI. ## Decisiones técnicas Justifica las decisiones relevantes. ## Tests Indica: - tests ejecutados; - resultado; - cobertura si está disponible. ## Riesgos o bloqueos Especialmente cualquier dato que todavía deba acordarse con M1, M2 o M3. ## Diff final Resume solamente cambios relacionados con esta tarea. No afirmes haber implementado algo que solo haya quedado preparado.		